Tail-Latency Tuning of Linux for Low-Latency Trading

An Empirical Study of User-Space Mechanisms and Bayesian Optimisation, with a Design for Cloud Evaluation

Carlos Perez Andrade (Student Number 2500731)

Supervisor: Dr Vahid Rafe

Module 7CS077 Dissertation

A dissertation submitted in partial fulfilment of the requirements for the degree of

MSc Computer Science with Artificial Intelligence

University of Wolverhampton

October 2026

Contents

Declaration **6**

Abstract **7**

List of Abbreviations **8**

1 Introduction **9**
1.1 Context and Motivation . . . 9
1.2 Problem Statement . . . 9
1.3 Scope, Aim, and Contributions . . . 10

2 Literature Review **11**
2.1 Tail Latency and its Economic Significance . . . 11
2.2 Sources of Latency Variance in Linux . . . 11
2.3 Kernel-Resident Tuning Techniques . . . 11
2.4 Kernel Bypass and its Contemporary Limits . . . 12
2.5 Precision Time Protocol and Measurement Fidelity . . . 12
2.6 Automated Search over Tuning Configurations . . . 13
2.7 Identified Research Gap . . . 13

3 Research Aims and Questions **14**
3.1 Aim and Objectives . . . 14
3.2 Research Questions . . . 14
3.3 Hypotheses . . . 14

4 Methodology **16**
4.1 Research Design . . . 16
4.2 Experimental Platform . . . 16
4.3 Workload and Instrumentation . . . 16
4.4 Tuning Mechanisms and Search Space . . . 17
4.5 Experimental Phases . . . 17
4.6 Search Procedures . . . 18
4.7 Simulation Study . . . 18
4.8 Metrics and Statistical Analysis . . . 19
4.9 Legal, Social, Ethical, and Professional Issues . . . 19

5 Implementation **21**
5.1 Real-Measurement Harness . . . 21
5.2 Simulation Code and Testing . . . 21

6 Results and Analysis **23**
6.1 The Untuned Baseline . . . 23
6.2 Effect of Each Mechanism (RQ1) . . . 23
6.3 Search on the Real Host (RQ2) . . . 25
6.4 Follow-up: Why Isolation Hurt . . . 28
6.5 Simulation Study (RQ2 and RQ3) . . . 28

7 Discussion **32**
7.1 Interpretation of Findings 32
7.2 Implications for Practice 33
7.3 Limitations 33

8 Conclusion **35**
8.1 Summary of Findings 35
8.2 Contributions 35
8.3 Recommendations for Future Work 35
8.4 Concluding Remarks 36

References **37**

Appendix A: Planned Cloud Protocol **42**

Appendix B: Supplementary Tables **44**

Appendix C: Repository Contents **49**

Appendix D: Project Management and Deviations **50**

List of Figures

1 Figure 6.1: Factorial phase: the p99.9 of every individual run, with the median of each configuration marked by a bar. Grey points are SCHED_OTHER and green points SCHED_FIFO. 24

2 Figure 6.2: Best observed p99.9 against the number of measurements, median over five seeds with the range across seeds shaded. The dashed line is the median per-run p99.9 of the untuned default in Phase A. 26

3 Figure 6.3: Complementary cumulative distribution of wake-up latency, pooled over the five re-evaluation runs, for the untuned default and for the best re-measured pick of each method. The dotted line marks the 99.9th percentile. 27

4 Figure 6.4: Regret of the picks of each method against the exact optimum of the synthetic model, median and interquartile range over 30 seeds. The dotted line marks a regret of 5 per cent, the threshold used in the text and in Table B.4. 29

5 Figure 6.5: Vargha-Delaney A12 of Bayesian optimisation against random search for every completed model, by budget. Points are the 56 one-at-a-time perturbations, boxes their quartiles, and the diamond the unperturbed base model. Values above 0.5 favour Bayesian optimisation; the dashed line marks no difference and the dotted line a large effect (0.71). 30

6 Figure 6.6: Importance rank of each parameter across the completed models. Left: exact total-effect Sobol index. Right: SHAP values of a random forest fitted to the random-search trials. 31

7 Figure A.1: Planned cloud topology. Solid lines indicate data-plane traffic; dashed lines indicate PTP synchronisation. 42

8 Figure A.2: Schematic of the intended CPU, IRQ, and NUMA layout of the consumer instance under the fully optimised cloud scenario (S5). The receiving thread is labelled “Producer” in the diagram; the order emitter shares the isolated SCHED_FIFO set with the strategy thread, and cores 0 and 1 remain with the operating system. Core counts are illustrative. 43

List of Tables

1 Table 4.1: Search space of the real-measurement study. 17

2 Table 6.1: Main effects on the logarithm of the per-run p99.9 (36 runs), as multiplicative factors with 95 per cent intervals; “sibling also freed” is the additional effect on a core that is already isolated. 24

3 Table B.1: Factorial phase (microseconds). Percentiles are the median over three runs; the pooled p99.9 is computed from the merged histograms with a run-level bootstrap interval, which with three runs approximates the range of the runs. “Max > 20,000” means at least one sample overflowed the histogram. 44

4 Table B.2: C-state follow-up on CPU 5 (microseconds), five runs per condition. Percentiles and maxima are medians over runs. The last column is the mean rate at which the measured CPU entered an idle state of C1E or deeper. 44

5 Table B.3: Re-evaluation (microseconds) of the untuned default and of every pick of Phase B, five runs each in shuffled order. Percentiles are medians over runs. Cliff’s delta compares all samples of the pick with all samples of the default; minus one means every sample of the pick was faster. 45

6 Table B.4: Budget study on the synthetic model, 30 seeds per cell. “Within 5%” is the fraction of seeds whose pick had a true 99.9th percentile no more than 5 per cent above the optimum. A12 is the probability that a pick of Bayesian optimisation has lower regret than a pick of random search; p is the one-sided Mann-Whitney test. . . 46

7 Table B.5: Coefficients of the synthetic latency model. “Magnitude” and “effect” name the two perturbation rules of the sensitivity study; “fixed” marks the two structural thresholds that are not perturbed. The untuned default has p99.9 = 766.7 microseconds and the optimum of the 2,879,712-point space has p99.9 = 10.3 microseconds. 46

8 Table B.6: Coverage of the one-at-a-time sensitivity sweep. Each cell gives, for the model in which one coefficient is perturbed by the factor f, the exact optimum p99.9 in microseconds and the lowest Vargha-Delaney A12 of Bayesian optimisation against random search over the budgets of 20 or more evaluations. The sweep ran the levels in the order 0.5, 2, 0.75, 1.5 and was stopped after 56 models, so the two extreme perturbations are complete for every coefficient and the 32 missing models are the milder ones. The lowest A12 in the table is 0.87 (iso_decay at f = 0.5, 0.865 before rounding); the base model has an optimum of 10.3 microseconds. 48

9 Table D.1: Planned milestones against the actual record. Dates before 13 October come from correspondence and the module calendar; dates from 13 October onwards come from git commits and experiment log timestamps. 50

Declaration

I declare that this dissertation is my own work and that it has not been submitted, in whole or in part, for any other degree or qualification at this or any other institution. Where the work of others has been used, it is acknowledged and referenced in the text.

Signed: *Carlos Perez Andrade*

Carlos Perez Andrade

Date: 18 October 2026

Abstract

In high-frequency trading the 99th and 99.9th percentiles of latency, not the mean, govern profitability, and timestamp accuracy is a regulatory obligation. A tuned Linux kernel is a credible alternative to kernel bypass, particularly in the public cloud, but choosing the tuning configuration is itself an expensive search problem: the mechanisms interact, each evaluation is a noisy measurement of a rare event, and in practice the configuration is set by hand.

This dissertation studies that problem on a commodity Linux workstation, as a feasible first step towards a cloud study it outlines. It measures the wake-up latency of a periodic thread under heavy contention in 401 measurements of 30 seconds each: a factorial design over real-time scheduling, affinity-based core isolation, and freeing of the simultaneous multithreading (SMT) sibling, with CPU pinning assessed observationally, and a comparison of Bayesian optimisation with random search in which every selected configuration is re-measured independently. A simulation with an exactly known optimum quantifies how the advantage of Bayesian optimisation depends on the measurement budget and tests it under perturbation.

Real-time scheduling reduced the 99.9th percentile by a factor of about 32 and separated the scheduling classes completely. Freeing the SMT sibling of an isolated core reduced it by a further 45 per cent. Isolation by affinity alone raised the 99th percentile about six-fold under the default scheduler; a follow-up experiment linked the isolation penalty to processor idle states. Every selected configuration cut the 99.9th percentile from about 2.5 milliseconds to between 14 and 69 microseconds, far beyond the four-fold target fixed in advance. Bayesian optimisation wasted far fewer measurements on poor configurations than random search and its picks had a lower median tail, though not significantly so with five seeds. In simulation it came within 5 per cent of the optimum with 20 evaluations in 83 per cent of seeds, whereas random search reached that in only half of its seeds at 150, and the advantage held under every completed perturbation. The results argue for model-based search with mandatory re-measurement, and indicate that core isolation should be paired with idle-state control.

List of Abbreviations

AWS: Amazon Web Services

BO: Bayesian Optimisation

CPU: Central Processing Unit

DPDK: Data Plane Development Kit

EC2: Amazon Elastic Compute Cloud

ENA: Elastic Network Adapter

HFT: High-Frequency Trading

IRQ: Interrupt Request

MiFID II: Markets in Financial Instruments Directive II

NIC: Network Interface Card

NTP: Network Time Protocol

NUMA: Non-Uniform Memory Access

PTP: Precision Time Protocol

RTS: Regulatory Technical Standard

SCHED_FIFO: First-In, First-Out real-time scheduling policy of Linux

SCHED_OTHER: Default time-sharing scheduling policy of Linux

SHAP: SHapley Additive exPlanations

SMT: Simultaneous Multithreading

UTC: Coordinated Universal Time

1 Introduction

1.1 Context and Motivation

In electronic markets, trading firms compete on latencies measured in microseconds. Hasbrouck and Saar (2013), analysing NASDAQ order data from 2007 and 2008, found low-latency responses clustering at two to three milliseconds, and reaction times have compressed by orders of magnitude since. At this scale the mean says little and the tail matters more, because, as Dean and Barroso (2013) showed for large-scale services, a rare slow response becomes a common one at scale. For a market-making engine a single outlier of a few hundred microseconds during a volatility spike can invalidate a live quote, expose the firm to adverse selection, or corrupt software timestamps that Regulatory Technical Standard 25 under MiFID II requires to be accurate to within 100 microseconds (European Commission, 2017). The quantity that matters is a tight, measurable bound on the 99th and 99.9th percentiles of tick-to-trade latency under representative load.

1.2 Problem Statement

The Linux kernel exhibits latency variance that is acceptable for general-purpose workloads and unacceptable for latency-critical ones; documented sources include interrupt coalescing, softirq deferral, frequency scaling, scheduler migration, NUMA memory-controller contention, and the periodic scheduling-clock interrupt (Dashti *et al.*, 2013; Herzog *et al.*, 2018; Bristot de Oliveira *et al.*, 2020). The established industry response is kernel bypass: frameworks such as DPDK, RDMA, and AF_XDP dedicate a network interface to one application, trading protection and portability for deterministic packet delivery (Belay *et al.*, 2014; Majkowski, 2015).

Two developments motivate re-examining this trade-off. First, engineering evidence suggests that a carefully tuned kernel recovers much of the throughput that bypass promises (Richards, 2022), and io_uring keeps kernel abstractions while removing most context-switch overhead (Wei and Begunkov, 2023), although at the highest line rates bypass remains clearly ahead (Jasny *et al.*, 2025). Second, precise time synchronisation is now a cloud service: AWS exposes a PTP Hardware Clock to guest instances with microsecond accuracy (Amazon Web Services, 2024; Karaoglu *et al.*, 2024), which makes one-way latency between cloud machines measurable with a clock independent of the system under test.

A further problem is how a configuration is chosen: the mechanisms interact, each evaluation is a noisy measurement of a rare event, and in practice the configuration is set by hand from checklists (Intel Corporation, 2024; Databento, 2024). Whether automated search can do better with few measurements has not, to the author's knowledge, been tested for kernel tail latency, nor has the interaction of cloud-provided time synchronisation with kernel-resident tuning been characterised

in peer-reviewed work.

1.3 Scope, Aim, and Contributions

The question that motivates this work is whether kernel-level tuning, combined with cloud-provided PTP synchronisation, can approach the tail-latency behaviour of kernel bypass in a high-frequency trading context. Answering it in full requires bare-metal cloud instances with a PTP Hardware Clock, a real-time kernel, and many instance-hours of measurement, none available to this project, so the dissertation takes a narrower step. Its aim is to establish, on a commodity Linux workstation under controlled contention, how much of the scheduling-latency tail can be removed by the tuning mechanisms that need no kernel reconfiguration, and whether a good combination of those mechanisms can be found automatically within a small measurement budget and confirmed by independent re-measurement. The workload is synthetic, for reasons of reproducibility and research ethics set out in Section 4.9, and the design of the cloud study is outlined in Appendix A.

The dissertation makes four contributions: a per-mechanism characterisation on real hardware of the tail-latency effect of each user-space mechanism and its dependence on scheduling policy, from full latency histograms; a comparison of Bayesian optimisation and random search as tuning procedures in which every selected configuration is re-measured, so that lucky measurements do not bias the comparison; a simulation study with an exactly known optimum, which quantifies how the advantage of Bayesian optimisation depends on the budget and tests that conclusion under perturbation; and a reproducible measurement harness, tests, and raw logs, published with this dissertation.

Chapter 2 reviews the literature and states the research gap, Chapter 3 the aim, questions, and hypotheses, Chapter 4 the methodology, Chapter 5 the implementation, Chapter 6 the results, Chapter 7 the discussion, and Chapter 8 the conclusions and future work.

2 Literature Review

2.1 Tail Latency and its Economic Significance

Dean and Barroso (2013) observed that when a request fans out to N parallel subsystems, each slow with probability p, the probability that at least one is slow is $1-(1-p)^N$, so modest per-component tail probabilities become near-certain tail events at scale and the mean misleads. In high-frequency trading the effect is amplified, because a slow response is picked off by faster participants, and Hasbrouck and Saar (2013) established, on NASDAQ data now nearly two decades old, that low-latency activity is a structural feature of equity markets. Regulation reinforces the point: RTS 25 under MiFID II requires high-frequency algorithmic traders to synchronise business clocks to UTC within 100 microseconds (European Commission, 2017). The quantities of interest are therefore the 99th and 99.9th percentiles of tick-to-trade latency under representative load, which also bound the accuracy of software timestamps. This dissertation measures their scheduling component.

2.2 Sources of Latency Variance in Linux

Boyd-Wickizer *et al.* (2010) showed that kernel lock contention in the scheduler, the virtual file system, and the memory allocator limits scalability on many-core systems, so latency variance is often an emergent property of contention, although their 2010 measurements predate much of the lock work in later kernels. On NUMA hardware, Dashti *et al.* (2013) showed that congestion on memory controllers, not remote-access delay, explains most variance, so pinning memory close to a core is necessary but not sufficient. At the interrupt level, Herzog *et al.* (2018) showed on ARM and Intel platforms that interrupt handlers and user-space workloads interact to produce jitter. Most pertinent is the PREEMPT_RT scheduling-latency bound of Bristot de Oliveira *et al.* (2020), which expresses worst-case latency as the sum of blocking and interference terms. Its value is that it applies to a deployed kernel; its limitation is that a worst-case bound and an empirical percentile are different quantities and cannot validate each other.

2.3 Kernel-Resident Tuning Techniques

Several mechanisms are well documented individually but rarely evaluated in controlled combination. CPU isolation through cpuset removes housekeeping work from designated cores; IRQ affinity steers interrupts away from the application thread; nohz_full removes the periodic scheduling-clock interrupt on isolated cores (Intel Corporation, 2024; Databento, 2024); and PREEMPT_RT, mainlined in Linux 6.12 (Torvalds, 2024), converts most spin-locks into preemptible mutexes and forces threaded interrupt handling (Bristot de Oliveira *et al.*, 2020). The evidence that these techniques work is largely grey literature: an engineering account of a cryptocurrency trading system reports a 99th-percentile matching latency under 25 microseconds at over 100,000 messages per second on one

pinned thread without kernel bypass (Liu, 2026), but such reports describe one system on one machine without a controlled baseline and almost never vary the techniques together or in virtualised environments, which motivates a factorial study.

Two hardware effects interact with these mechanisms and are often left out of tuning guides. The first is simultaneous multithreading (SMT), under which two hardware threads share one physical core's execution units and caches (Tullsen, Eggers and Levy, 1995), so a latency-critical thread can be slowed by work on its sibling even when it has its logical CPU to itself. The second is processor idle states: a core with nothing to run enters a C-state in which parts of it are powered down, and waking from a deep state takes from a few to hundreds of microseconds. Schöne, Molka and Werner (2015) measured these latencies on x86 processors against the vendor specifications exposed to the operating system, and real-time practice therefore limits idle states on latency-critical cores (Intel Corporation, 2024). An isolated core spends more of its time idle, so isolation and idle states are not independent, a point that becomes central in Chapter 6.

2.4 Kernel Bypass and its Contemporary Limits

Kernel-bypass frameworks dedicate a network interface to one application through poll-mode drivers, removing interrupt and system-call overhead at the cost of an always-busy core, vendor-specific drivers, loss of standard observability, and limited cloud-provider support (Belay *et al.*, 2014; Majkowski, 2015). The evidence on the size of the gap is mixed. Jasny *et al.* (2025) show that on 400 Gbit hardware neither sockets nor io_uring can saturate the link with small messages and conclude that bypass remains necessary at that scale, whereas Richards (2022), in a non-peer-reviewed benchmark, found that kernel tuning narrowed DPDK's throughput advantage for an HTTP workload from about 4.2 times to about 1.5 times, and Wei and Begunkov (2023) present zero-copy receive through io_uring as removing much of the remaining overhead. The disagreement is partly one of regime, since a trading host handles a feed at tens of thousands of messages per second, not at line rate. Where the tuned kernel stands in that regime under cloud constraints is the long-term question of this line of work, which the present dissertation does not measure.

2.5 Precision Time Protocol and Measurement Fidelity

PTP, standardised as IEEE 1588-2019, uses hardware timestamping under assumptions of path symmetry (IEEE, 2019; Byagowi and Obleukhov, 2022). It improves the accuracy of latency measurement, not latency itself: with NTP-based timestamps Flow Traders observed apparent one-way latencies on AWS as low as minus 56 microseconds, an artefact eliminated by the PTP Hardware Clock, although the account is a vendor case study without independent replication (Karaoglu *et al.*, 2024; Amazon Web Services, 2024). In the planned cloud study (Appendix A), PTP is instrumentation, not an intervention; the local study measures on a single clock and does not need

it.

2.6 Automated Search over Tuning Configurations

The mechanisms above interact, and once a handful of them is combined the configuration space grows to thousands of points, each measured for many seconds before its tail percentiles can be trusted. Bayesian optimisation is designed for this setting: an objective that is costly to evaluate, observed with noise, and without useful gradients. A Gaussian-process surrogate is fitted to the evaluations so far, and an acquisition function such as expected improvement trades off exploitation against exploration (Jones, Schonlau and Welch, 1998; Snoek, Larochelle and Adams, 2012; Shahriari *et al.*, 2016). In systems research the same machinery has tuned database knobs (Van Aken *et al.*, 2017), cloud configurations for analytics jobs (Alipourfard *et al.*, 2017), and distributed systems, where Dalibard, Schaarschmidt and Yoneki (2017) found that a plain Gaussian process degrades beyond about ten dimensions (the spaces here have five and eight). Across storage systems no black-box tuner was best everywhere (Cao *et al.*, 2018), whereas on held-out machine-learning problems Bayesian methods beat random search (Turner *et al.*, 2021). Random search is the natural control, because it is simple to parallelise and hard to beat when only a few parameters matter (Bergstra and Bengio, 2012).

This literature also carries two cautions. A surrogate-guided search reports its best observed value, which under noise is optimistically biased, so the selected configuration must be re-measured before its performance is claimed; CherryPick describes no such step. And conclusions drawn from one objective may not survive another, which argues for testing under perturbed assumptions, with variance-based sensitivity indices (Sobol', 2001) measuring how much each matters. Neither caution appears to have been applied to kernel tail-latency tuning.

2.7 Identified Research Gap

From this review, the evidence for kernel tuning is largely grey literature from single bare-metal systems without controlled baselines, which limits its applicability to cloud-first engineering. The formal latency-bound literature has been validated on real-time kernels, not on virtualised servers; the literature on cloud PTP consists largely of vendor case studies; and although automated tuning has reached databases, storage, and cloud instance choice, kernel latency parameters are still set by hand from checklists, even though the space is large, the mechanisms interact, and every evaluation is a noisy and expensive tail measurement. No study known to the author has varied these mechanisms together against a controlled baseline or asked whether their configuration can be searched systematically with a small budget; this dissertation addresses both gaps on commodity hardware.

3 Research Aims and Questions

3.1 Aim and Objectives

The long-term aim of this line of work is to characterise and bound the tail latency attainable by a kernel-resident, PTP-synchronised Linux system in a public-cloud, HFT-adjacent context. Within this project's resources, the aim is to establish on commodity Linux hardware how much of the scheduling-latency tail the mechanisms that need no kernel reconfiguration can remove, and whether a good combination of them can be found automatically with a small budget and confirmed by re-measurement. Five objectives serve this aim: (1) build a reproducible harness that records the full wake-up latency distribution of a periodic thread under controlled contention and can be driven by a search procedure; (2) measure the effect of real-time scheduling, affinity-based core isolation, and freeing the SMT sibling, singly and in combination, and of CPU pinning observationally; (3) compare Bayesian optimisation against random search with independent re-measurement of the selected configurations; (4) use a synthetic latency model with a known optimum to quantify how many measurements each procedure needs and test that conclusion under perturbation; and (5) outline the design of the cloud study (Appendix A).

3.2 Research Questions

RQ1. On a commodity Linux host without a real-time kernel, how much do real-time scheduling, CPU pinning, affinity-based isolation, and freeing the SMT sibling reduce the 99th and 99.9th percentiles of scheduling latency under contention, individually and in combination?

RQ2. Can Bayesian optimisation identify a near-optimal configuration within a small measurement budget, and does it select better configurations than random search when every evaluation is a noisy tail-latency measurement?

RQ3. Does the advantage of Bayesian optimisation, and the ranking of parameter importance, hold when the coefficients of the latency model are perturbed?

The proposal's questions on cloud PTP timestamping, the analytical scheduling-latency bound, and the residual gap to kernel bypass require the cloud platform and a real-time kernel and remain the agenda of the cloud study.

3.3 Hypotheses

The proposal fixed a quantitative threshold for success before any data were collected, and it is kept unchanged.

H1. The best configuration found reduces the 99.9th-percentile scheduling latency by at least a

factor of four relative to the untuned default, as measured by independent re-evaluation. The threshold was set for tick-to-trade latency in the cloud, so it is a minimum standard rather than a demanding test.

H2. For equal budgets on the real host, the configurations selected by Bayesian optimisation are at least as good as those selected by random search when re-measured. A re-measured disadvantage, a Vargha-Delaney A12 below 0.5 for the picks of Bayesian optimisation against those of random search, refutes H2. The proposal stated H2 qualitatively; this criterion was made explicit after data collection. It is descriptive: if the two methods were equivalent it would be met by chance about half the time, so a result that does not refute H2 is weak evidence in its favour.

4 Methodology

4.1 Research Design

The study combines two executed strands and one specified but unexecuted strand. The first is a controlled empirical study on a commodity Linux workstation, in which every data point is a real measurement of scheduling latency under contention; it answers RQ1 and provides the real-measurement test of RQ2. The second is a simulation study on a synthetic latency model whose optimum is known exactly; it says nothing about real kernels, but it can score a search procedure against a known answer, which no real measurement can, and so answers RQ2 in that sense and RQ3. The third is the cloud study proposed originally (Appendix A). The executed strands constrain each other: the simulation budget study, completed first, set the budget spent on the real host, and the real measurements show which of the simulation's assumptions hold on hardware. A controlled experiment was chosen over observation of a production system because the research questions concern causal effects of individual mechanisms. The stance is post-positivist: latency effects are treated as real and measurable, but every estimate is reported with its uncertainty and the conditions under which it was obtained, since one host and one kernel cannot establish general laws.

4.2 Experimental Platform

All real measurements were taken on one workstation: an Intel Core i7-8700 (six cores, twelve hardware threads, one NUMA node), 30 GiB of memory, Ubuntu 26.04.1 LTS with the generic 6.17 kernel (PREEMPT_DYNAMIC, 1000 Hz tick, not PREEMPT_RT), no isolcpus, nohz_full, or rcu_nocbs boot parameters, irqbalance not running, and no frequency-scaling driver exposed to user space. No experiment ran as root; the only privilege was a raised RLIMIT_RTPRIO of 99, which lets an unprivileged process request SCHED_FIFO. The restriction is deliberate: it limits the study to mechanisms an application team can apply without changing the kernel or its boot parameters, as in managed environments, and keeps the experiment safe to run unattended.

4.3 Workload and Instrumentation

The latency-critical component of a tick-to-trade path is a thread that must run as soon as an event arrives. The quantity measured is that thread's wake-up latency, the delay between the moment a timer was due to fire and the moment the thread runs, measured with cyclictest 2.80 from the rt-tests suite (The Linux Foundation Real-Time Linux Project, 2024). One thread wakes every 200 microseconds, so a 30-second measurement yields about 150,000 samples, recorded in a histogram with 1-microsecond bins up to 20 milliseconds; samples beyond the last bin are treated as right-censored. This measures only the scheduling component of the critical path, on a single clock, so no external time synchronisation is needed.

Contention is created with stress-ng (King, 2024): one matrix-multiplication worker on each CPU the configuration allows, plus two memory workers each touching 256 MiB, started two seconds before the measurement and stopped after it; the load is identical across configurations except where one deliberately keeps it off some CPUs. Before and after each measurement the harness records the interrupt and local-timer rates of every CPU from /proc/interrupts, with the load average, to help identify the mechanism behind a change in the tail.

4.4 Tuning Mechanisms and Search Space

Five parameters define a configuration (Table 4.1). Equivalent combinations are normalised before measuring (priority is irrelevant under SCHED_OTHER, isolation is meaningless for an unpinned thread, and sibling exclusion applies only when the core is isolated), leaving 3,700 distinct configurations: 37 under SCHED_OTHER and 3,663 under SCHED_FIFO. Isolation is achieved in user space by restricting the affinity of the background load, not with isolcpus, so it removes user-space competition from the measured core but leaves kernel threads, timers, and interrupts there. The untuned default is SCHED_OTHER, unpinned, with the load free to run anywhere.

Table 4.1: Search space of the real-measurement study.

Parameter	Values	Mechanism
policy	SCHED_OTHER or SCHED_FIFO	Scheduling class of the measured thread
rt_priority	1 to 99	SCHED_FIFO priority (ignored under SCHED_OTHER)
cpu	unpinned, or CPU 0 to 11	CPU affinity of the measured thread
isolate	off or on	Background load kept off the measured CPU
sibling_free	off or on	Background load also kept off the SMT sibling

4.5 Experimental Phases

The real study runs in four phases of 30-second measurements. Phase A (baseline) measures the untuned default ten times. Phase B (search) lets Bayesian optimisation and random search each choose 30 configurations for five seeds, interleaved so that slow drift affects both equally; each method's pick for a seed is the configuration with the lowest observed 99.9th percentile. Phase C (re-evaluation) measures every pick and the default five more times in shuffled order, because the minimum of 30 noisy observations is partly the luckiest draw; only these measurements are used to compare methods or claim an improvement. Phase D (factorial) measures twelve configurations three times each in shuffled order: both policies (SCHED_FIFO at priority 80), three isolation

levels (none, core isolated, core and sibling free), and two CPUs (CPU 0, which handles more device interrupts by default, and CPU 5). The study comprises 401 measurements, about four hours; a 35-measurement follow-up testing one unexpected result is reported in Section 6.4.

4.6 Search Procedures

The objective is the base-10 logarithm of the measured 99.9th percentile, because tail latencies span several orders of magnitude and a Gaussian process models the logarithm far better. The surrogate is the Gaussian process regressor of scikit-optimize (Head *et al.*, 2021), version 0.10.2, with the same settings in both studies. Each configuration is mapped to a unit cube (integers scaled to 0 to 1, binary parameters as 0 or 1). The kernel is a constant amplitude multiplied by an anisotropic Matérn kernel of smoothness 5/2, one length scale per dimension, plus a white-noise term (Snoek, Larochelle and Adams, 2012); amplitude, length scales, and noise level are re-fitted by maximising the marginal likelihood after every observation, from two restarts, on the standardised objective. The next configuration maximises expected improvement over the best observed value (Jones, Schonlau and Welch, 1998) with an exploration offset of 0.01: 10,000 random candidates are scored, the best five refined with L-BFGS-B, and the result rounded to the grid. Each seed starts from eight configurations drawn uniformly at random, since seven hyperparameters cannot be fitted from fewer observations, and the model proposes the remaining 22. The budget of 30 evaluations per seed follows from the simulation budget study (Section 6.5), which placed the separation between the two methods at 20 to 40 evaluations, and from measurement time, since two methods and five seeds already account for 300 of the 401 measurements. Random search draws each configuration independently and uniformly from the same unit cube, under the same budget, seeds, and interleaving (Bergstra and Bengio, 2012; Turner *et al.*, 2021), so it chooses SCHED_FIFO with probability one half; the methods differ only in how the next point is chosen, so any difference between them is attributable to the surrogate and acquisition function, not to budget, bounds, or noise. Noise enters the model only through the white-noise term, so the best observed value is still partly the luckiest draw, which is why Phase C re-measures every pick.

4.7 Simulation Study

The simulation replaces measurement with a synthetic model in which each request has a log-normal body latency and, with some probability, a Pareto-distributed stall. Eight tuning parameters (isolated cores, IRQ affinity, nohz_full, PREEMPT_RT, frequency governor, NUMA pinning, real-time priority, and interrupt coalescing) act through 22 coefficients chosen by hand to mimic the qualitative behaviour reported in the literature (Table B.5). Because the model is explicit, its exact 99.9th percentile can be computed for every one of the 2,879,712 configurations, so every pick is scored by its regret, the ratio of its true 99.9th percentile to the optimum, minus one. Each noisy

evaluation draws one million samples. The budget study runs both methods at budgets of 10, 20, 40, 80, and 150 evaluations, 30 seeds each; Bayesian optimisation draws its first 10 points at random, so at a budget of 10 it is itself a random search. The sensitivity study perturbs each coefficient one at a time by factors of 0.5, 0.75, 1.5, and 2 (88 models; rules in Table B.5), with a planned joint sweep of 128 models along a scrambled Sobol sequence (Sobol', 1967), and repeats the budget study on every model.

4.8 Metrics and Statistical Analysis

The primary metrics are the 50th, 99th, and 99.9th percentiles of wake-up latency (p50, p99, p99.9) and the maximum. The statistical unit is one 30-second measurement, because samples within a measurement are serially dependent. A configuration is summarised by the median of its runs' percentiles and by the percentile of the pooled histogram of its runs, with a 95 per cent interval from a bootstrap over whole runs (5,000 resamples); with three runs the interval is indicative only. Differences against the default are summarised by Cliff's delta (Cliff, 1993). The picks of the two methods are compared with the Vargha-Delaney A12 statistic (Vargha and Delaney, 2000), which alone decides whether H2 is refuted, and with a one-sided Mann-Whitney U test (Mann and Whitney, 1947) for the advantage H2 anticipates. The factorial phase is analysed with a main-effects linear model on the logarithm of p99.9, checked with a lack-of-fit test, so that each coefficient is a multiplicative effect on the tail. Tests use a significance level of 0.05. In the simulation, regret replaces latency and parameter importance is measured by exact total-effect Sobol indices (Sobol', 2001), with SHAP values of a random forest fitted to the random-search trials (Lundberg and Lee, 2017) as the estimate an analyst without the true model could make. A Kolmogorov-Smirnov test was not used because with 150,000 autocorrelated samples per run it flags trivial differences.

4.9 Legal, Social, Ethical, and Professional Issues

Ethically, the workload is entirely synthetic: no market data, order entry, or exchange connection is involved, so there is no risk of market impact, and no human participants or personal data are implicated. The project therefore falls outside research ethics review, and the Ethical Considerations Form was submitted at proposal stage with all questions answered negatively. The experiments ran on the author's own workstation without administrative privileges and affected no other users; a SCHED_FIFO thread at priority 99 could in principle starve a CPU, but the measured thread sleeps for most of each 200-microsecond period and the kernel's default real-time throttling remained in place.

Legally, the tools used are open source, and the repository publishes the code under the MIT licence and the text and figures under CC BY 4.0. The clock-accuracy rules of MiFID II (European Commission, 2017) motivate the work but impose no obligation on a laboratory study that records

no business events. Socially, lower tail latency serves an industry whose benefits are contested, but the same mechanisms govern telemetry, control, audio, and payment systems, and the published harness lets those communities reuse the method. Professionally, the study follows the BCS Code of Conduct in working within the author's competence, reporting unexpected and negative results (Section 6.4) as fully as positive ones, and making every number in Chapter 6 regenerable from the published code and raw logs (Appendix C).

5 Implementation

Everything is written in Python 3.14 against NumPy, SciPy, scikit-learn, and scikit-optimize, and kept in one public repository with the raw measurement logs (Appendix C), so that each figure and table can be regenerated by one script. The tick-to-trade pipeline and cloud provisioning of Appendix A have not been implemented.

5.1 Real-Measurement Harness

The harness (scripts/experiments/real_pilot.py) turns a configuration into one measurement. It normalises the configuration, computes the CPU set the background load may use, reads the SMT sibling from the kernel's topology files, starts stress-ng under taskset, waits two seconds, reads the interrupt counters, runs cyclictest with the requested affinity and policy, reads the counters again, and stops the load by signalling its process group so that no worker survives into the next measurement. Each measurement is appended as one JSON record with the configuration, percentiles, full histogram, per-CPU interrupt rates, load average, and timestamp. On start-up the harness skips every measurement already in the log, so an interrupted run resumes where it stopped and repeating a command never overwrites data. The search driver uses the ask-and-tell interface of scikit-optimize, so the Gaussian process is updated after every real measurement, and runs the later phases in a seeded shuffled order, so the order of the 401 measurements is reproducible. An earlier pilot with a synthetic Python trading loop had measured queueing delay instead of scheduling jitter because its offered load exceeded the service rate, which is why cyclictest, which has no queue, was adopted.

5.2 Simulation Code and Testing

The synthetic model lives in bo_sim.py; bo_budget.py adds the exact ground truth by solving the mixture distribution numerically and verifying the optimum by enumeration, and bo_sensitivity.py generalises this to perturbed models, saving the optimiser's state between invocations. An early version drew only 20,000 samples per evaluation, at which the 99.9th percentile rests on about 20 tail samples and the best observed configuration was largely the luckiest draw; the reported studies use one million samples, and the same lesson motivated Phase C of the real study. Unit tests cover the normalisation rules, the CPU sets given to the load, the cyclictest and stress-ng command lines, histogram parsing, overflows, and percentile extraction, including a fixture for a floating-point error in the nearest-rank computation found before any data were collected; simulation tests check determinism, the direction of the main modelled effects, the agreement between exact and Monte Carlo percentiles, and that a search chain saved and resumed in segments reproduces an unsegmented one. A smoke test takes two real three-second measurements and checks that no

stress-ng worker is left behind, and a miniature run of all four phases preceded the four-hour run; both need a raised RLIMIT_RTPRIO.

6 Results and Analysis

The real-measurement study ran for 3.6 hours on 15 October 2026 and completed all 401 planned measurements. Every number below comes from the summaries of the analysis script of Section 5.2 or was recomputed from the raw logs by scripts in the repository. Latencies are in microseconds; OTHER denotes SCHED_OTHER and FIFO followed by a number denotes SCHED_FIFO at that priority. Full per-configuration tables are in Appendix B.

6.1 The Untuned Baseline

The ten baseline measurements of Phase A collected 1,437,824 wake-up samples. The median of every run was 56 microseconds, about 0.27 per cent of wake-ups took longer than one millisecond, the pooled p99.9 was 2,926 microseconds (95 per cent bootstrap interval 2,858 to 3,002), and two samples exceeded the 20-millisecond histogram range; the per-run p99.9 had a coefficient of variation of about 6 per cent. Re-measured in Phase C about three hours later, the default had a pooled p99.9 of 2,541 microseconds (2,159 to 2,780), a drift far smaller than the effects below.

6.2 Effect of Each Mechanism (RQ1)

Figure 6.1 shows every run of the factorial phase, Table B.1 the per-configuration percentiles, and Table 6.1 the main-effects model on the logarithm of the per-run p99.9. A lack-of-fit test does not reject the model for p99.9 ($F(7, 24) = 1.46$, $p = 0.23$) but rejects it for p99 ($F(7, 24) = 23.7$, $p < 0.001$), because the effect of isolation on p99 differs between policies, so p99 is described per policy.

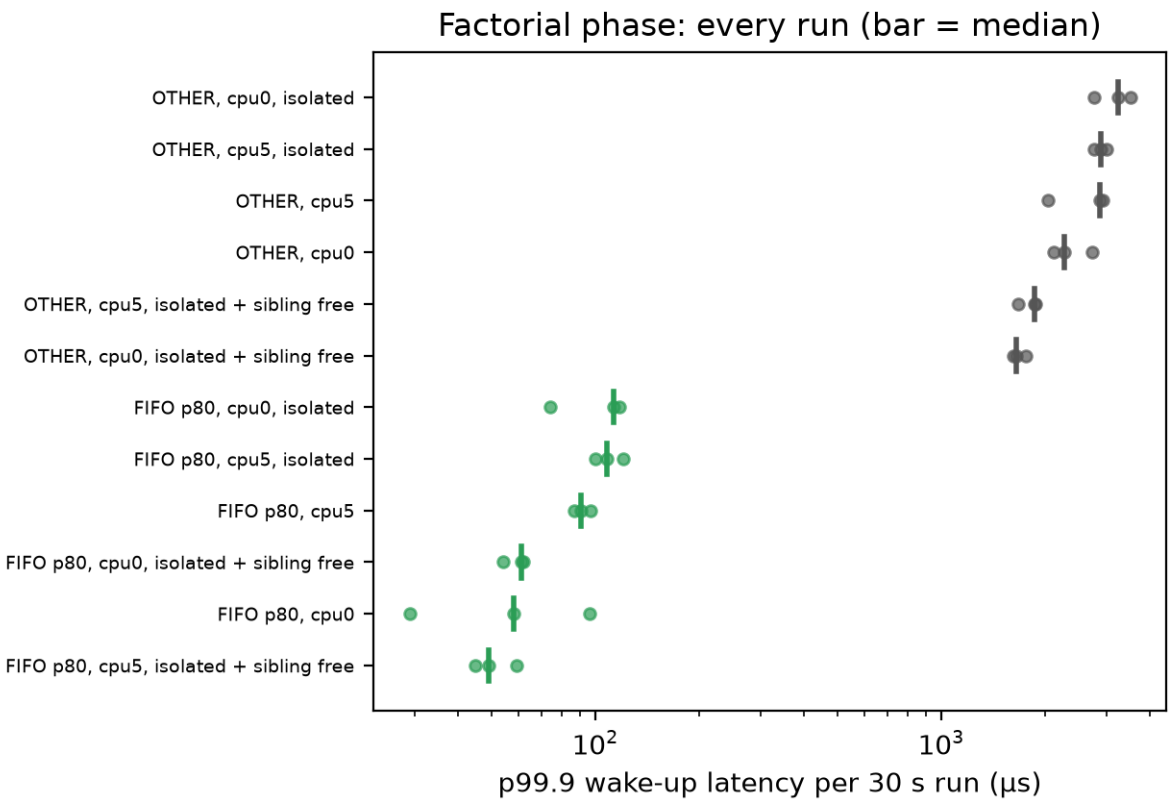


Figure 6.1: Factorial phase: the p99.9 of every individual run, with the median of each configuration marked by a bar. Grey points are SCHED_OTHER and green points SCHED_FIFO.

Table 6.1: Main effects on the logarithm of the per-run p99.9 (36 runs), as multiplicative factors with 95 per cent intervals; “sibling also freed” is the additional effect on a core that is already isolated.

Mechanism	Factor on p99.9	95 per cent interval
SCHED_FIFO (priority 80)	0.031	0.027 to 0.036
Core isolated by affinity	1.34	1.12 to 1.61
Sibling also freed	0.55	0.46 to 0.66
CPU 5 instead of CPU 0	1.09	0.94 to 1.26

The real-time scheduling class dominates. SCHED_FIFO divided the p99.9 by about 32 and the p99 by between 6 and 27 depending on isolation level. The separation is complete across the whole study: all 239 SCHED_FIFO measurements had a p99.9 of at most 124 microseconds and all 162 SCHED_OTHER measurements at least 1,616. SCHED_FIFO also removed the 56-microsecond floor, lowering the median wake-up to 5 or 6 microseconds.

Isolating the core by affinity, on its own, made the tail worse. Under SCHED_OTHER the median p99 rose from about 80 to about 480 microseconds, about six-fold; under SCHED_FIFO from 11 to 18; and the p99.9 rose by a factor of 1.34 with an interval that excludes one. This is the opposite of what the proposal assumed and motivated the follow-up of Section 6.4. Isolation did cut the worst case under SCHED_FIFO from 3,835 and 9,388 microseconds to below 1,000.

Freeing the SMT sibling helped the extreme tail and was the only form of isolation that did: relative to isolating the core alone it multiplied the p99.9 by 0.55, and the best factorial configuration, SCHED_FIFO on CPU 5 with core and sibling free, reached a pooled p99.9 of 50 microseconds. The p99 behaved differently. Under SCHED_OTHER freeing the sibling lowered it from about 480 to about 210 microseconds, but under SCHED_FIFO it raised it from 18 to about 34, which Section 6.4 links to idle states.

The choice of CPU made no detectable difference: the measured CPU handled 5,200 to 6,800 interrupts per second in every configuration, almost all local-timer interrupts, and device interrupts were negligible on this idle-network workstation. Pinning itself, assessed observationally from Phases B and C, made no significant difference either (median p99.9 of 23 microseconds unpinned against 33 pinned on a shared core among SCHED_FIFO measurements; two-sided Mann-Whitney U, $p = 0.36$), although the searches chose these measurements, so the comparison is indicative only.

6.3 Search on the Real Host (RQ2)

Each method spent 30 measurements per seed, five seeds each, on the 3,700-configuration space (Figure 6.2). In the median over seeds both methods found a configuration with an observed p99.9 below 20 microseconds within about 10 measurements, and after 30 the median best observed value was 11 microseconds for Bayesian optimisation and 15 for random search. The budgets were spent very differently: over measurements 9 to 30 of every seed, after its eight random initial points, Bayesian optimisation proposed SCHED_FIFO in 85 per cent of measurements with a median p99.9 of 54 microseconds, whereas random search proposed SCHED_FIFO in 43 per cent with a median of 1,820. Every one of the ten picks uses SCHED_FIFO and none uses core isolation.

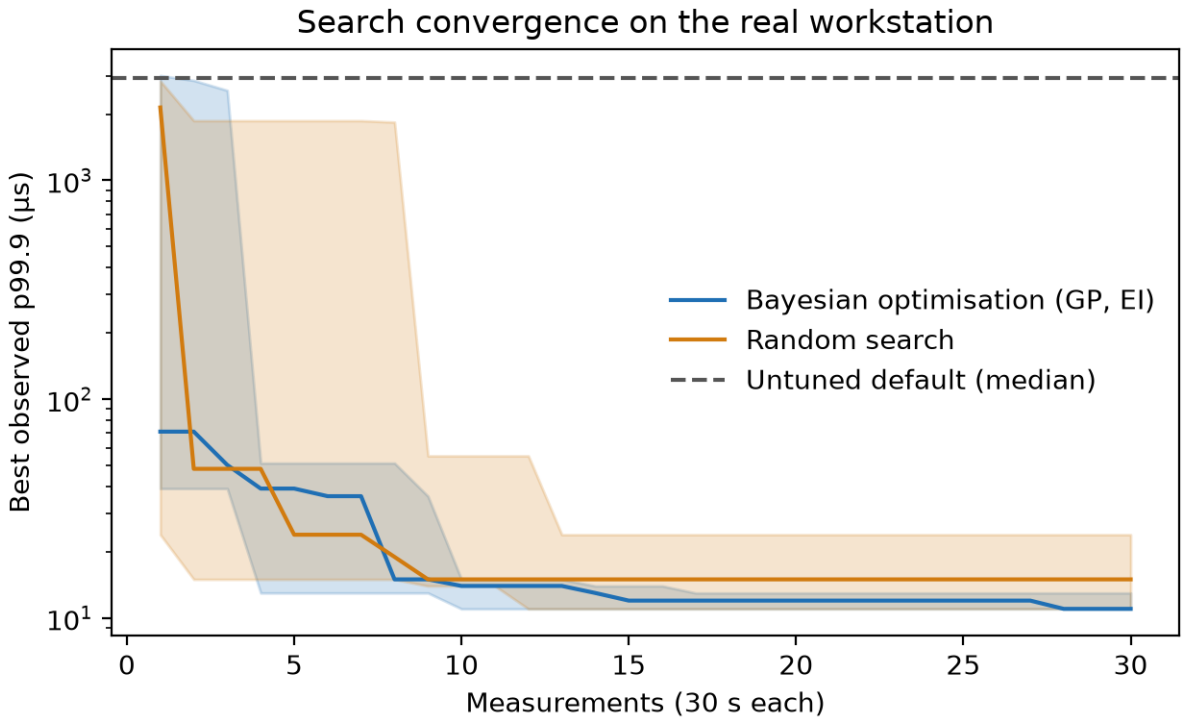


Figure 6.2: Best observed p99.9 against the number of measurements, median over five seeds with the range across seeds shaded. The dashed line is the median per-run p99.9 of the untuned default in Phase A.

Best observed values are biased downwards, which is why Phase C re-measured every pick five times (Table B.3; Figure 6.3 compares the tail of each method's best re-measured pick with the default). Re-measurement confirmed the picks but not their values: the re-measured median p99.9 was 19 microseconds for Bayesian optimisation and 61 for random search, against best observed medians of 11 and 15, and several pooled intervals span 15 to 85 microseconds because the tails consist of rare bursts that only some runs contain. A 30-second p99.9 thus ranks configurations reliably when they differ by an order of magnitude, but not when they differ by a factor of two.

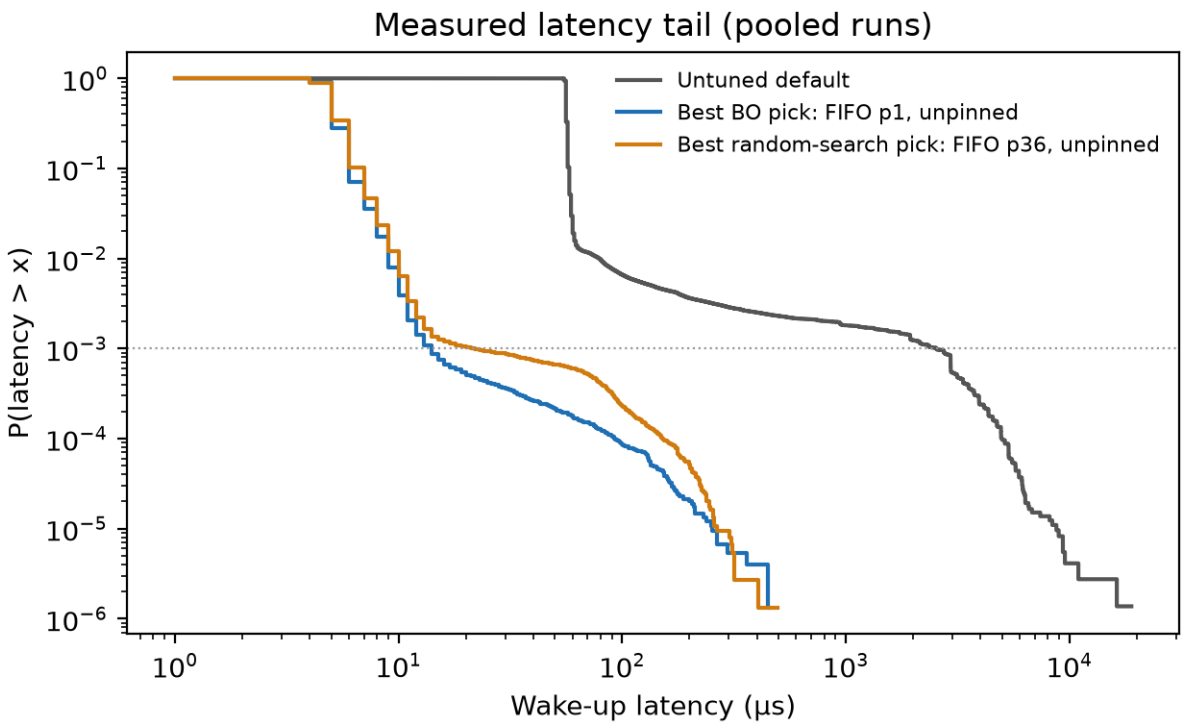


Figure 6.3: Complementary cumulative distribution of wake-up latency, pooled over the five re-evaluation runs, for the untuned default and for the best re-measured pick of each method. The dotted line marks the 99.9th percentile.

H1 is supported. Against the default re-measured in the same phase, every one of the ten picks reduced the pooled p99.9 by a factor of at least about 37 and the median pick by about 49; the best re-measured pick (SCHED_FIFO at priority 1, unpinned) reached 14 microseconds (12 to 17), although choosing the best of ten picks reintroduces some selection effect. Cliff's delta is at most minus 0.997 for every pick: almost every wake-up of a tuned configuration was faster than almost every wake-up of the default.

H2 is not refuted. The Vargha-Delaney statistic for the re-measured picks is A12 = 0.78 in favour of Bayesian optimisation, so no disadvantage was observed, but the advantage is not significant with five seeds per method (one-sided Mann-Whitney U, p = 0.09). The comparison is inconclusive rather than negative: the difference in median p99.9 is 42 microseconds (95 per cent bootstrap interval minus 6 to 52), and at the observed spread five seeds can detect a factor of about 2.4 in the geometric mean of p99.9 at 80 per cent power, whereas the observed ratio of geometric means was 2.0, which eight seeds per method would be expected to detect (real_power.py). Under the descriptive criterion of Section 3.3 this is weak evidence.

6.4 Follow-up: Why Isolation Hurt

The adverse effect of isolation was not anticipated, so a 35-measurement follow-up was run after the main study with the same harness and contention. The hypothesis was that an isolated core falls idle between wake-ups, enters processor idle states (C-states), and pays their exit latency on every wake-up, which the kernel's cpuidle interface reports as 2 to 890 microseconds for this processor; without root cyclictest cannot hold the power-management latency target at zero. The test adds a keep-awake process, a busy loop pinned to the measured CPU under SCHED_IDLE, which keeps the core out of idle states but yields at once to any other runnable task, and records how often the CPU entered each idle state. Five runs of each of seven conditions were measured in shuffled order on CPU 5 (Table B.2).

The results support the explanation: a core shared with the load went idle about five times per second, whereas isolated with its sibling free it entered an idle state more than 6,000 times per second, after almost every wake-up, mostly the shallow C1E and C3 states. The keep-awake process reduced this to zero and the penalty disappeared: under SCHED_FIFO the p99 returned from 33 to 14 microseconds, close to the 12 of the shared core, and under SCHED_OTHER the p99 fell from 272 to 111 and the pooled p99.9 from about 2,180 to about 990. With idle states removed, an isolated core with a free sibling matched a shared core at the median and p99 and had a lower extreme tail (pooled p99.9 of 53 against 93 microseconds under SCHED_FIFO), although with five runs that difference is not significant. The keep-awake process could also change other per-core behaviour, such as the idle tick, so the experiment supports the explanation without ruling out every alternative, and it tested only the core-and-sibling-free condition. A check of timer slack as the cause of the 56-microsecond floor of SCHED_OTHER was negative.

6.5 Simulation Study (RQ2 and RQ3)

The synthetic model has an exact optimum of 10.29 microseconds against 766.67 for its default configuration. Figure 6.4 and Table B.4 report the regret of the picks over 30 seeds per budget. At a budget of 10 both methods are random searches and the difference ($A12 = 0.61$, $p = 0.065$) reflects only different random draws. From 20 evaluations onwards the difference is large and consistent (A12 of 0.96 or higher, $p < 0.001$): Bayesian optimisation reaches within 5 per cent of the optimum in 83 per cent of seeds at 20, in every seed from 40, and finds the exact optimum in 77 per cent of seeds at 150, whereas the median regret of random search is still 4.9 per cent at 150.

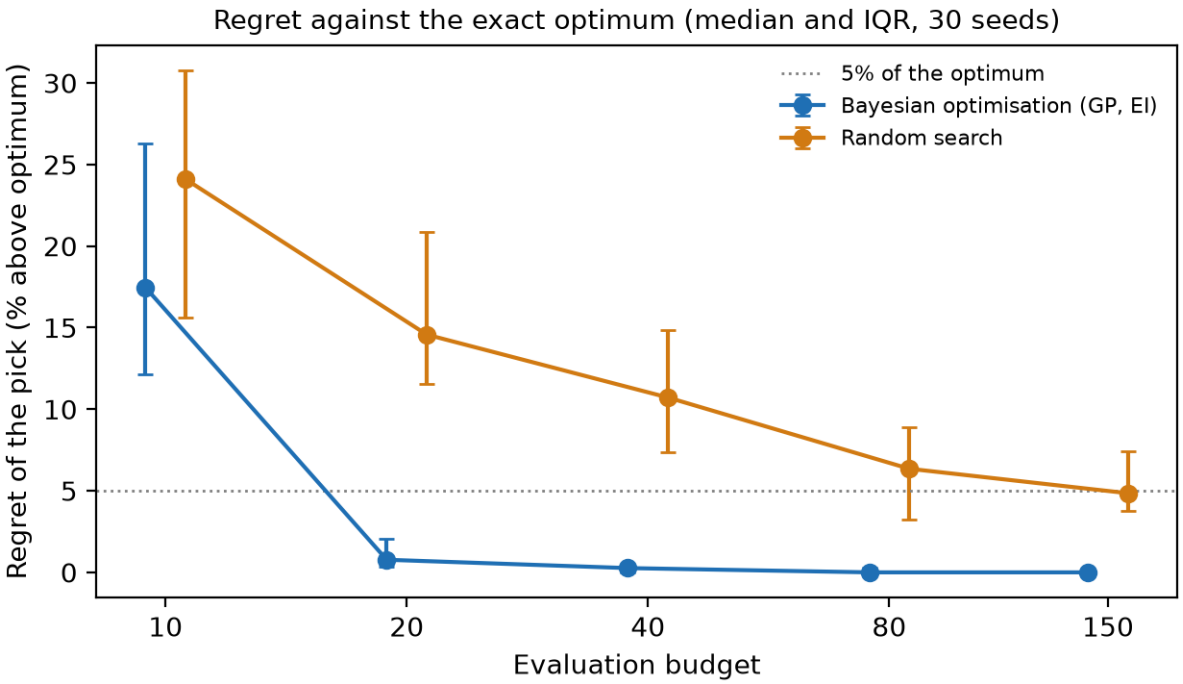


Figure 6.4: Regret of the picks of each method against the exact optimum of the synthetic model, median and interquartile range over 30 seeds. The dotted line marks a regret of 5 per cent, the threshold used in the text and in Table B.4.

The sensitivity study was stopped, under the priority rule of Appendix D, after the base model and 56 of the 88 one-at-a-time perturbations. The sweep ran the levels in the order 0.5, 2, 0.75, 1.5, so both extreme perturbations of all 22 coefficients are complete and the 32 missing models are the milder ones (Table B.6); the joint Sobol sweep was not started, so the results describe robustness to one changed assumption at a time, not to interactions between them. Across the 57 models the optimum ranged from 5.1 to 22.5 microseconds, so the perturbations changed the problem substantially. The conclusion survived every one of them (Figure 6.5): at every budget of 20 or more Bayesian optimisation was significantly better in every model, with a lowest A12 of 0.87, although all models share the same 30 seeds, so the result is stability under each single perturbation rather than 57 independent confirmations.

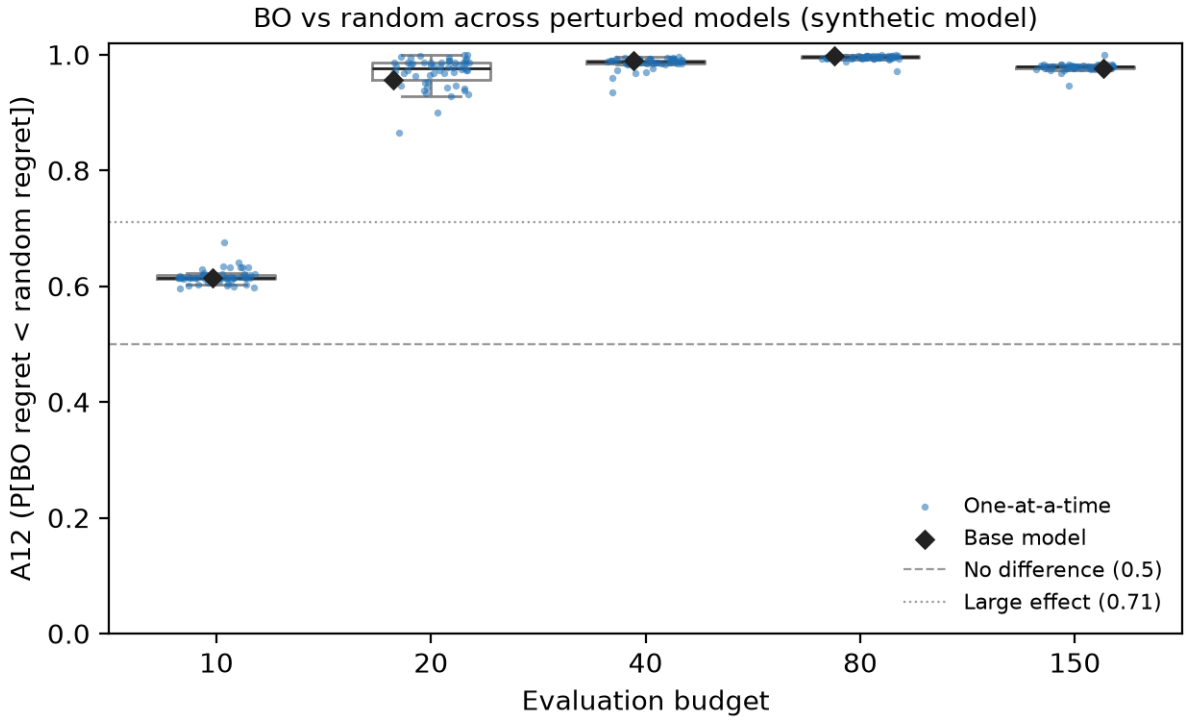


Figure 6.5: Vargha-Delaney A12 of Bayesian optimisation against random search for every completed model, by budget. Points are the 56 one-at-a-time perturbations, boxes their quartiles, and the diamond the unperturbed base model. Values above 0.5 favour Bayesian optimisation; the dashed line marks no difference and the dotted line a large effect (0.71).

The ranking of parameter importance was also stable (Figure 6.6): the number of isolated cores was the most important parameter by exact total-effect Sobol index in all 57 models, nohz_full second in 89 per cent, and real-time priority last in 91 per cent, and SHAP rankings estimated from the random-search trials alone ranked isolated cores first in all 57 models and real-time priority last in 56, agreeing with the exact ranking of the least important parameter in 51. The synthetic model makes core isolation its dominant parameter whereas the real study found isolation by affinity harmful on its own; Section 7.1 explains why the two are compatible.

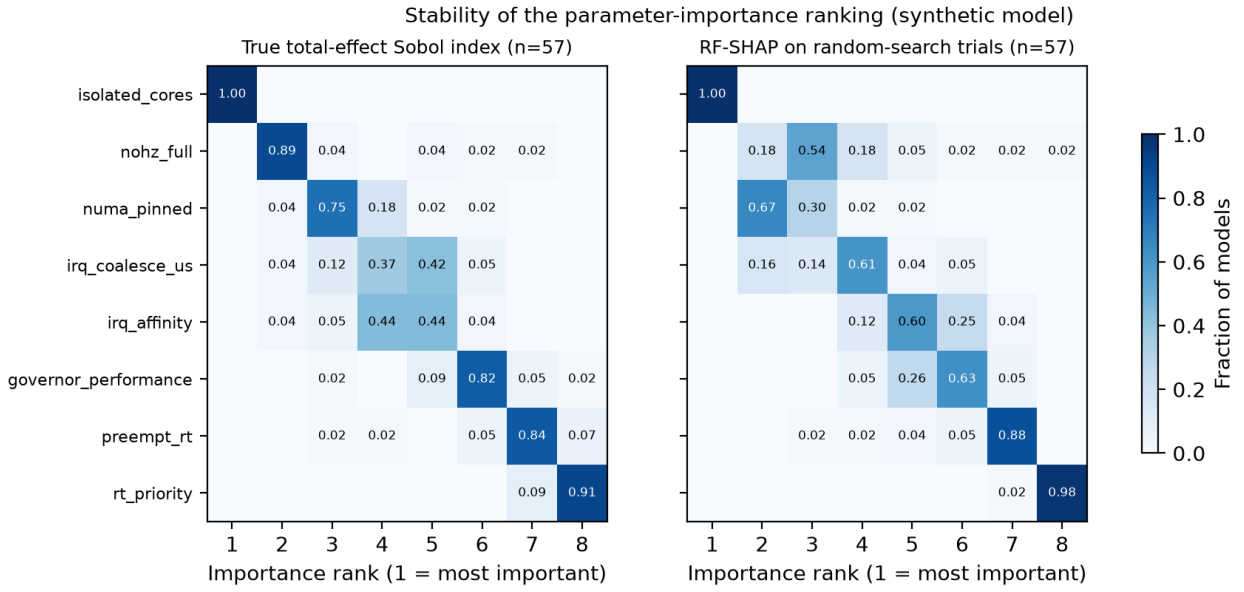


Figure 6.6: Importance rank of each parameter across the completed models. Left: exact total-effect Sobol index. Right: SHAP values of a random forest fitted to the random-search trials.

7 Discussion

7.1 Interpretation of Findings

On this platform the untuned tail is caused mainly by scheduling. Under SCHED_OTHER the measured thread competes with the load on equal terms and occasionally waits milliseconds for a preemption point; SCHED_FIFO removes that competition because a runnable real-time thread preempts every SCHED_OTHER task at once. The remaining tail, tens to a few hundred microseconds, is what the scheduler cannot remove: non-preemptible kernel sections, interrupt handling, and interference through shared hardware. This matches the decomposition of Bristot de Oliveira *et al.* (2020), in which the latency of the highest-priority thread is bounded by blocking from non-preemptible sections and interference from interrupts; on a non-real-time kernel those sections are unbounded, which is why PREEMPT_RT is still needed to bound the worst case. It also explains why the priority value made no difference (the picks in Table B.3 span priorities 1 to 99): the measured thread was the only real-time thread.

The adverse effect of isolation needs the most explanation. Practitioner guides treat core isolation as a first step in latency tuning (Databento, 2024), but they mean isolation at boot combined with measures that keep the core out of idle states, and isolation by affinity achieves only the first half. Once the load is removed the core idles between wake-ups and pays the exit latency measured by Schöne, Molka and Werner (2015) on the next one; here the entries were mostly to shallow states, but they followed almost every wake-up, so even exit latencies of tens of microseconds reached the 99th percentile. Isolation is therefore not harmful in itself: it moves the bottleneck from the scheduler to power management, and only a configuration that addresses both, for example through the kernel's power-management latency target (The Linux Kernel Organization, 2026), delivers the expected benefit. The SMT result fits the same picture: a compute-heavy worker on the sibling slows the measured thread through shared execution units and caches (Tullsen, Eggers and Levy, 1995), and removing it reduced the 99.9th percentile by 45 per cent, a control that the account of Liu (2026) does not mention; but a physical core can only enter its deeper idle states when both hardware threads are idle, so freeing the sibling also removed the protection a loaded sibling had provided, and the 99th percentile rose under SCHED_FIFO. The larger penalty of isolation alone under SCHED_OTHER was not tested in the follow-up and is attributed to idle states by extension. The simulation, calibrated to a literature that describes boot-time isolation with idle-state control, is reconciled with the real result in the same way.

The real search comparison and the simulation agree once the structure of the two spaces is considered. In the real space one binary parameter accounts for almost all the variation, so a random sample of 10 configurations contains a SCHED_FIFO configuration with probability $1 - 2^{-10}$, and every SCHED_FIFO configuration measured already exceeded the H1 threshold; the remaining

differences among SCHED_FIFO configurations are of the order of the run-to-run variation of one configuration, which is why five seeds could not separate the methods. The simulation shows the same pattern at larger scale: random search reliably samples the dominant parameter but leaves a median regret of 5 to 15 per cent because it does not refine the rest. The practical advantage of Bayesian optimisation lay in how the budget was spent, 15 per cent of its measurements after initialisation on SCHED_OTHER configurations against 57 per cent for random search; when each measurement costs minutes of machine time or hours of cloud instance time, that economy is the main benefit, which agrees with CherryPick (Alipourfard *et al.*, 2017) and with Bergstra and Bengio (2012) on the competitiveness of random search when few parameters matter.

Re-measurement changed the picture as the early simulation had suggested: best observed medians of 11 and 15 microseconds became 19 and 61, the optimistic bias noted in Section 2.6, so a tuning procedure should always end by re-measuring its choice.

7.2 Implications for Practice

Run the latency-critical thread under a real-time scheduling class; on this machine that one change removed about 97 per cent of the 99.9th-percentile latency and needs only a raised RLIMIT_RTPRIO. Do not isolate a core by affinity alone: keep it out of idle states, through the power-management latency target or busy polling, or leave it shared. Keep the SMT sibling of the critical core free of heavy work, with idle-state control. When tuning automatically, prefer a model-based search for its economy of measurements and always re-measure the selected configuration. The regulatory dimension is only partly addressed: RTS 25 constrains timestamp accuracy, not latency (European Commission, 2017), and this study made no PTP measurement, but a single wake-up delayed by more than 20 milliseconds, as observed even in a tuned run, is far beyond the 100-microsecond tolerance, so bounding the tail is necessary for credible software timestamping but not sufficient.

7.3 Limitations

Six limitations bound these conclusions. The metric is scheduling latency, one component of tick-to-trade latency. The study used one workstation, one processor model, and a generic kernel; idle-state latencies, SMT behaviour, and interrupt routing differ between processors, so the magnitudes do not transfer directly to servers or cloud instances, although the direction of the main effects more likely does. The experiment ran without root, so the power-management latency target, IRQ affinity, isolcpus, nohz_full, and PREEMPT_RT were unavailable. The contention was synthetic, heavier and more uniform than that of a production host, so the measured effects are probably larger than production would show. Five seeds per method were enough to show that both methods meet the success criterion but not to separate them. Finally, the simulation's model was calibrated by hand

and its sensitivity study tested only one-at-a-time perturbations, with the milder levels incomplete. Each limitation is addressed in Section 8.3.

8 Conclusion

8.1 Summary of Findings

This dissertation began from the question of how far kernel-resident tuning, with cloud-provided precision time, can approach the tail latency of kernel bypass in a high-frequency trading context, and took the narrower step its resources allowed: 401 measurements on a commodity Linux workstation under heavy contention, a short explanatory experiment, and a simulation study with an exactly known optimum.

On RQ1, real-time scheduling is by far the most important mechanism on this platform: SCHED_FIFO reduced the 99.9th-percentile wake-up latency by a factor of about 32 and separated the scheduling classes completely. Freeing the SMT sibling of an isolated core reduced it by a further 45 per cent, whereas isolating the core by affinity alone raised the 99th percentile about six-fold under SCHED_OTHER; the follow-up experiment linked the isolation penalty to idle-state entry. On RQ2, both search methods found configurations with an observed 99.9th percentile below 20 microseconds within about 10 measurements; Bayesian optimisation wasted far fewer measurements on poor configurations and its re-measured picks had a median 99.9th percentile of 19 microseconds against 61 ($A12 = 0.78$), not significant with five seeds. H1 was supported and H2 was not refuted. In simulation, Bayesian optimisation came within 5 per cent of the optimum in 83 per cent of seeds with 20 evaluations, whereas random search reached that in only half of its seeds at 150, and on RQ3 that advantage held under every completed one-at-a-time perturbation.

8.2 Contributions

The dissertation contributes a per-mechanism account of the tail latency of a commodity Linux host under contention, based on full histograms and a balanced factorial design, which shows that isolation by affinity alone, the user-space stand-in for the boot-time isolation that practitioner guides recommend, raised the 99th percentile when the core was free to enter idle states. It also contributes a comparison of Bayesian optimisation and random search with independent re-measurement of every pick; a simulation study with an exact optimum and a perturbation analysis; and the published harness, tests, and raw logs from which every result can be regenerated.

8.3 Recommendations for Future Work

The limitations of Section 7.3 lead to six extensions, ordered by how directly they follow from the results. First, the power-management latency target and the deepest permitted idle state should become search parameters, which needs root but no kernel change. Second, kernel-level mechanisms

(isolcpus, nohz_full, rcu_nocbs, IRQ affinity, PREEMPT_RT) should be added to the search space one at a time. Third, more seeds per method at the same budget: eight would resolve the factor of two observed in Section 6.3, and twenty, about eleven hours of measurement, a factor of 1.5. Fourth, the remaining sensitivity models can be resumed from the saved checkpoints and the joint Sobol sweep started with the same command. Fifth, the cloud study of Appendix A would answer the questions this dissertation deferred: measurement fidelity with a PTP Hardware Clock, the analytical bound, and the residual gap to kernel bypass. Sixth, replaying an archived exchange feed in place of the synthetic load would test whether the ranking of mechanisms survives bursty, correlated arrivals.

8.4 Concluding Remarks

The main practical lesson is that the largest gains in tail latency on a commodity Linux host are cheap and quick to find: one setting, the real-time scheduling class, removed about 97 per cent of the 99.9th-percentile latency, and either search procedure found it within minutes. Two results qualify this. Isolating a core by affinity made the 99th percentile worse unless the core was kept out of idle states, and the best observed value of a search understated the re-measured tail of its pick by a median factor of about 4 for random search and 1.7 for Bayesian optimisation. Re-measuring the selected configuration, checking mechanisms with a factorial design, and testing the search on a model with a known optimum were all needed to reach these conclusions; the cloud study should keep all three.

References

Alipourfard, O. *et al.* (2017) "CherryPick: Adaptively Unearthing the Best Cloud Configurations for Big Data Analytics," *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pp. 469-482.

Amazon Web Services (2024) "Amazon Time Sync Service Expands Microsecond-Accurate Time to 87 Additional EC2 Instance Types." AWS What's New, https://aws.amazon.com/about-aws/whats-new/2024/04/amazon-time-sync-service-microsecond-accurate-time-additonal-ec2-instance-types.

Amazon Web Services (2026) "Set the Time Reference on Your EC2 Instance to Use the Local Amazon Time Sync Service." Amazon EC2 User Guide, https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/configure-ec2-ntp.html.

Belay, A. *et al.* (2014) "IX: A Protected Dataplane Operating System for High Throughput and Low Latency," *Proc. 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. Broomfield, CO, USA, pp. 49-65.

Bergstra, J. and Bengio, Y. (2012) "Random Search for Hyper-Parameter Optimization," *Journal of Machine Learning Research*, 13, pp. 281-305.

Boyd-Wickizer, S. *et al.* (2010) "An Analysis of Linux Scalability to Many Cores," *Proc. 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. Vancouver, BC, Canada, pp. 1-16.

Bristot de Oliveira, D. *et al.* (2020) "Demystifying the Real-Time Linux Scheduling Latency," *Proc. 32nd Euromicro Conference on Real-Time Systems (ECRTS)*. (LIPIcs), pp. 9:1-9:23.

Byagowi, A. and Obleukhov, O. (2022) "PTP: Timing Accuracy and Precision for the Future of Computing." Meta Engineering Blog, https://engineering.fb.com/2022/11/21/production-engineering/future-computing-ptp/.

Cao, Z. *et al.* (2018) "Towards better understanding of black-box auto-tuning: A comparative analysis for storage systems," *Proceedings of the 2018 USENIX annual technical conference (USENIX ATC '18)*. Boston, MA: USENIX Association, pp. 893-907.

Cliff, N. (1993) "Dominance Statistics: Ordinal Analyses to Answer Ordinal Questions," *Psycholog-*

ical Bulletin, 114(3), pp. 494-509.

Dalibard, V., Schaarschmidt, M. and Yoneki, E. (2017) "BOAT: Building auto-tuners with structured Bayesian optimization," *Proceedings of the 26th international conference on world wide web (WWW '17)*. Perth, Australia: International World Wide Web Conferences Steering Committee, pp. 479-488. Available at: https://doi.org/10.1145/3038912.3052662.

Dashti, M. *et al.* (2013) "Traffic Management: A Holistic Approach to Memory Placement on NUMA Systems," *Proc. ASPLOS*. Houston, TX, USA, pp. 381-394.

Databento (2024) "Low-Latency Tuning Guide for Linux and Trading Systems." Databento Blog, https://medium.databento.com/low-latency-tuning-guide-for-linux-and-trading-systems-by-databento-4e35eaa43f07.

Dean, J. and Barroso, L.A. (2013) "The Tail at Scale," *Communications of the ACM*, 56(2), pp. 74-80.

European Commission (2017) "Commission Delegated Regulation (EU) 2017/574 of 7 June 2016 supplementing Directive 2014/65/EU with regard to regulatory technical standards for the level of accuracy of business clocks." Official Journal of the European Union, L 87/148.

Hasbrouck, J. and Saar, G. (2013) "Low-Latency Trading," *Journal of Financial Markets*, 16(4), pp. 646-679.

Head, T. *et al.* (2021) "scikit-optimize/scikit-optimize, version 0.9.0." Zenodo. https://doi.org/10.5281/zenodo.5565057.

Herzog, B. *et al.* (2018) "INTspect: Interrupt Latencies in the Linux Kernel," *Proc. VIII Brazilian Symposium on Computing Systems Engineering (SBESC)*. Salvador, Brazil, pp. 113-120. Available at: https://doi.org/10.1109/SBESC.2018.00021.

IEEE (2019) "IEEE Standard for a Precision Clock Synchronization Protocol for Networked Measurement and Control Systems." IEEE Std 1588-2019.

Intel Corporation (2024) "Real-Time Scheduling on Linux." Intel Edge Controls for Industrial documentation, release 3.3, https://eci.intel.com/docs/3.3/.

Jasny, M. *et al.* (2025) "A Wake-Up Call for Kernel-Bypass on Modern Hardware," *Proc. 21st International Workshop on Data Management on New Hardware (DaMoN)*. Berlin, Germany. Available at: https://doi.org/10.1145/3736227.3736235.

Jones, D.R., Schonlau, M. and Welch, W.J. (1998) "Efficient Global Optimization of Expensive Black-Box Functions," *Journal of Global Optimization*, 13(4), pp. 455-492.

Karaoglu, S. *et al.* (2024) "Gaining Microsecond Advantage: How Amazon EC2's Precision Timekeeping Empowers Flow Traders." AWS for Industries Blog, https://aws.amazon.com/blogs/industries/gaining-microsecond-advantage-how-amazon-ec2s-precision-timekeeping-empowers-flow-traders.

King, C.I. (2024) "stress-ng: a tool to load and stress a computer system." https://github.com/ColinIanKing/stress-ng.

Liu, J.-H. (2026) "Design and Implementation of a Low-Latency High-Frequency Trading System for Cryptocurrency Markets." Medium, https://medium.com/@gwrx2005/design-and-implementation-of-a-low-latency-high-frequency-trading-system-for-cryptocurrency-markets-a1034fe33d97.

Lundberg, S.M. and Lee, S.-I. (2017) "A Unified Approach to Interpreting Model Predictions," *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, pp. 4765-4774.

Majkowski, M. (2015) "Kernel Bypass." Cloudflare Blog, https://blog.cloudflare.com/kernel-bypass/.

Mann, H.B. and Whitney, D.R. (1947) "On a Test of Whether One of Two Random Variables Is Stochastically Larger than the Other," *The Annals of Mathematical Statistics*, 18(1), pp. 50-60.

Richards, M. (2022) "Linux Kernel versus DPDK: HTTP Performance Showdown." talawah.io Engineering Blog, https://talawah.io/blog/linux-kernel-vs-dpdk-http-performance-showdown/.

Schöne, R., Molka, D. and Werner, M. (2015) "Wake-up Latencies for Processor Idle States on Current x86 Processors," *Computer Science, Research and Development*, 30(2), pp. 219-227. Available at: https://doi.org/10.1007/s00450-014-0270-z.

Shahriari, B. *et al.* (2016) "Taking the Human Out of the Loop: A Review of Bayesian Optimization," *Proceedings of the IEEE*, 104(1), pp. 148-175.

Snoek, J., Larochelle, H. and Adams, R.P. (2012) "Practical Bayesian Optimization of Machine Learning Algorithms," *Advances in Neural Information Processing Systems 25 (NIPS 2012)*, pp. 2951-2959.

Sobol', I.M. (1967) "On the Distribution of Points in a Cube and the Approximate Evaluation of Integrals," *USSR Computational Mathematics and Mathematical Physics*, 7(4), pp. 86-112.

Sobol', I.M. (2001) "Global Sensitivity Indices for Nonlinear Mathematical Models and their Monte Carlo Estimates," *Mathematics and Computers in Simulation*, 55(1-3), pp. 271-280.

Tene, G. (2014) "HdrHistogram: A High Dynamic Range Histogram." Open-source software, https://github.com/HdrHistogram/HdrHistogram.

The Linux Foundation Real-Time Linux Project (2024) "cyclictest (rt-tests suite)." https://wiki.linuxfoundation.org/realtime/documentation/howto/tools/cyclictest/start.

The Linux Kernel Organization (2026) "PM Quality of Service Interface." The Linux Kernel documentation, https://www.kernel.org/doc/html/latest/power/pm_qos_interface.html.

Torvalds, L. (2024) "Linux 6.12-rc1." Linux kernel mailing list, via LWN.net, https://lwn.net/Articles/992184/.

Tullsen, D.M., Eggers, S.J. and Levy, H.M. (1995) "Simultaneous Multithreading: Maximizing On-Chip Parallelism," *Proc. 22nd Annual International Symposium on Computer Architecture (ISCA)*, pp. 392-403.

Turner, R. *et al.* (2021) "Bayesian optimization is superior to random search for machine learning hyperparameter tuning: Analysis of the black-box optimization challenge 2020," in H.J. Escalante and K. Hofmann (eds) *Proceedings of the NeurIPS 2020 competition and demonstration track*. PMLR (Proceedings of machine learning research), pp. 3-26. Available at: https://proceedings.mlr.press/v133/turner21a.html.

Van Aken, D. *et al.* (2017) "Automatic database management system tuning through large-scale machine learning," *Proceedings of the 2017 ACM international conference on management of data (SIGMOD '17)*. Chicago, IL: ACM, pp. 1009-1024. Available at: https://doi.org/10.1145/3035918.3064029.

Vargha, A. and Delaney, H.D. (2000) "A Critique and Improvement of the CL Common Language Effect Size Statistics of McGraw and Wong," *Journal of Educational and Behavioral Statistics*, 25(2), pp. 101-132.

Wei, D. and Begunkov, P. (2023) "Zero Copy Receive Using io_uring," *Proc. Netdev Conference 0x17*. Available at: https://netdevconf.info/0x17/sessions/talk/zero-copy-receive-using-io_uring.html.

Appendix A: Planned Cloud Protocol

The cloud study keeps the design of the original proposal and extends the harness developed in this dissertation. It would run on Amazon EC2 bare-metal instances of the C7i family, for which the Nitro PTP Hardware Clock is exposed to Linux guests as a PTP device (/dev/ptp_ena) when the instance is launched in a precision time placement group with ENA driver 2.10 or later (Amazon Web Services, 2024, 2026), and which support hardware packet timestamping at ingress on the Elastic Network Adapter. Bare-metal instances avoid a co-resident hypervisor scheduler while keeping the public-cloud setting in which the research gap was identified. A producer and a consumer instance would be placed in one placement group, with data-plane traffic and PTP synchronisation on separate paths (Figure A.1).

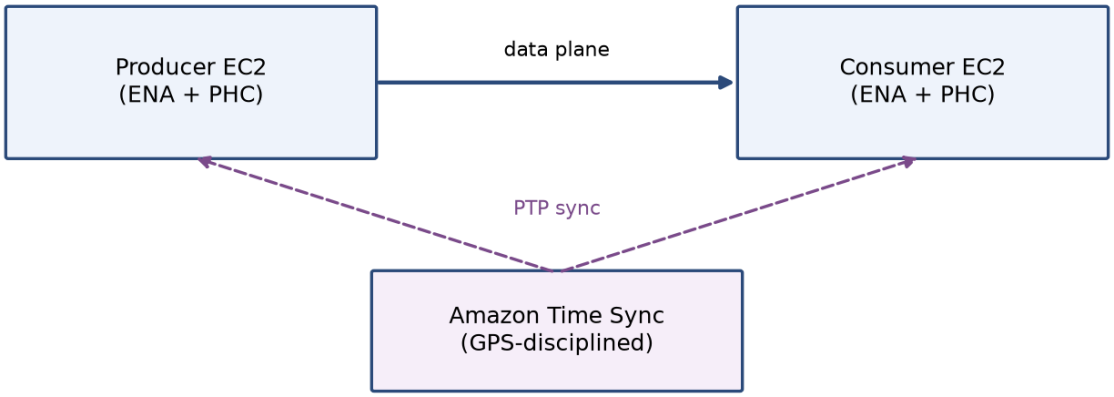


Figure A.1: Planned cloud topology. Solid lines indicate data-plane traffic; dashed lines indicate PTP synchronisation.

The workload is a three-stage synthetic tick-to-trade pipeline (market-data producer, strategy evaluator, order emitter) under Poisson arrivals at four offered loads, timestamped at NIC ingress, application read, post-decision, and NIC egress, and accumulated in HdrHistogram structures (Tene, 2014). Six scenarios form an interpretable gradient: S0 stock Ubuntu with irqbalance; S1 adds IRQ affinity; S2 adds cpuset isolation; S3 adds NUMA-aware allocation; S4 adds nohz_full on the isolated cores; S5 adds PREEMPT_RT with the strategy thread under SCHED_FIFO. Figure A.2 shows the intended CPU, IRQ, and NUMA layout for S5. The analytical strand instantiates the PREEMPT_RT scheduling-latency bound of Bristot de Oliveira *et al.* (2020) with interrupt rates measured by the osnoise tracer and compares the predicted worst case with the observed distribution, treating the two as different quantities rather than as validators of each other. The interrupt-rate

recording already built into the local harness is the first step towards that parameterisation.

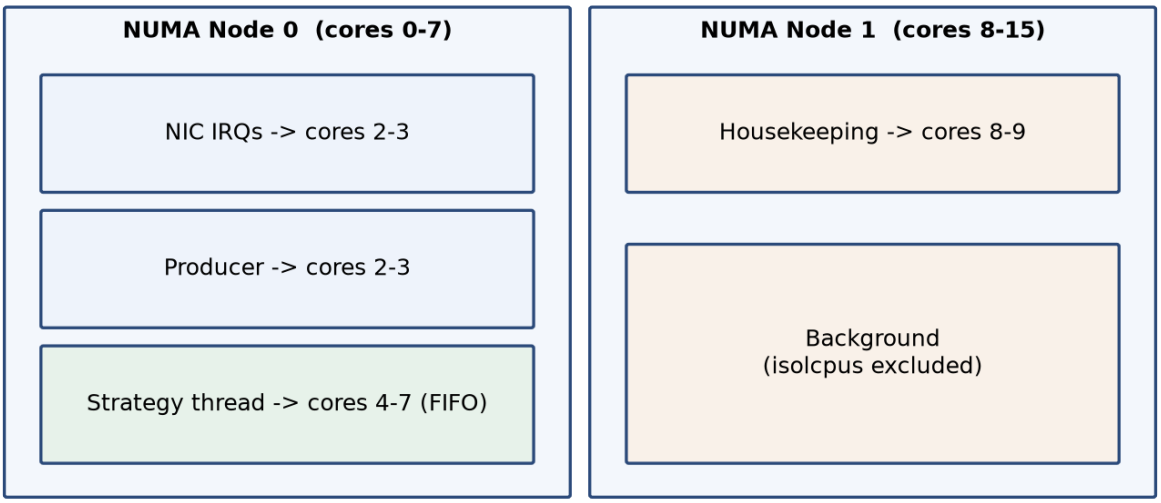


Figure A.2: Schematic of the intended CPU, IRQ, and NUMA layout of the consumer instance under the fully optimised cloud scenario (S5). The receiving thread is labelled "Producer" in the diagram; the order emitter shares the isolated SCHED_FIFO set with the strategy thread, and cores 0 and 1 remain with the operating system. Core counts are illustrative.

The present study changes one element of this plan. Instead of measuring only the fixed gradient S0 to S5, the cloud study should let the search procedure validated here choose among the mechanisms, with the gradient retained as a set of reference points. The budget study of Section 6.5 suggests, on the synthetic model, 20 to 40 evaluations per search. At five minutes per evaluation, ten seeds at each of the four offered loads would need roughly 70 to 130 instance-hours for each search method, or 130 to 270 hours to compare two methods, before re-measurement. The results of Chapter 6 add two requirements: the search space should include idle-state control from the start, and every selected configuration should be re-measured independently.

Appendix B: Supplementary Tables

Table B.1: Factorial phase (microseconds). Percentiles are the median over three runs; the pooled p99.9 is computed from the merged histograms with a run-level bootstrap interval, which with three runs approximates the range of the runs. "Max > 20,000" means at least one sample overflowed the histogram.

Configuration	p50	p99	p99.9	p99.9 pooled [95% CI]	Max
OTHER, CPU 0	56	105	2,264	2,458 [2,115, 2,732]	> 20,000
OTHER, CPU 5	56	70	2,865	2,642 [2,031, 2,928]	> 20,000
OTHER, CPU 0, isolated	57	531	3,251	3,189 [2,771, 3,513]	> 20,000
OTHER, CPU 5, isolated	57	444	2,888	2,890 [2,761, 3,009]	> 20,000
OTHER, CPU 0, isolated, sibling free	57	206	1,647	1,685 [1,616, 1,758]	> 20,000
OTHER, CPU 5, isolated, sibling free	56	213	1,861	1,803 [1,670, 1,866]	> 20,000
FIFO 80, CPU 0	5	11	58	77 [29, 96]	9,388
FIFO 80, CPU 5	5	11	91	90 [87, 97]	3,835
FIFO 80, CPU 0, isolated	6	20	113	102 [74, 117]	611
FIFO 80, CPU 5, isolated	6	14	108	109 [100, 120]	574
FIFO 80, CPU 0, isolated, sibling free	5	35	61	58 [54, 62]	642
FIFO 80, CPU 5, isolated, sibling free	6	33	49	50 [45, 59]	903

Table B.2: C-state follow-up on CPU 5 (microseconds), five runs per condition. Percentiles and maxima are medians over runs. The last column is the mean rate at which the measured CPU entered an idle state of C1E or deeper.

Condition	p50	p99	p99.9	p99.9 pooled [95% CI]	Max	Idle entries per second
FIFO 80, core shared with load	5	12	93	93 [50, 118]	2,358	5
FIFO 80, core and sibling free	7	33	66	65 [60, 71]	498	6,312
FIFO 80, core and sibling free, keep-awake	5	14	59	53 [38, 71]	481	0
OTHER, core and sibling free	57	272	1,935	2,176 [1,767, 2,668]	16,524	6,638

Condition	p50	p99	p99.9	p99.9 pooled [95% CI]	Max	Idle entries per second
OTHER, core and sibling free, keep-awake	54	111	830	985 [614, 1,328]	12,650	0
OTHER, core and sibling free, slack 1 ns	57	302	2,180	2,768 [1,903, 3,418]	18,844	6,151
OTHER, core and sibling free, keep-awake, slack 1 ns	54	115	895	817 [577, 950]	11,039	0

Table B.3: Re-evaluation (microseconds) of the untuned default and of every pick of Phase B, five runs each in shuffled order. Percentiles are medians over runs. Cliff's delta compares all samples of the pick with all samples of the default; minus one means every sample of the pick was faster.

Pick	Configuration	p50	p99	p99.9	p99.9 pooled [95% CI]	Max	Cliff's delta
Default	OTHER, unpinned	56	75	2,385	2,541 [2,159, 2,780]	18,913	0
BO, seed 0	FIFO 1, CPU 10	5	10	19	54 [14, 87]	453	-0.998
BO, seed 1	FIFO 52, CPU 10	5	10	17	17 [14, 23]	> 20,000	-0.999
BO, seed 2	FIFO 1, unpinned	5	10	27	50 [17, 80]	2,322	-0.998
BO, seed 3	FIFO 99, unpinned	5	10	31	58 [26, 79]	2,392	-0.998
BO, seed 4	FIFO 1, unpinned	5	9	14	14 [12, 17]	454	-1.000
Random, seed 0	FIFO 80, unpinned	5	10	67	59 [15, 87]	6,505	-0.998
Random, seed 1	FIFO 36, unpinned	5	10	17	21 [14, 60]	494	-0.999
Random, seed 2	FIFO 64, CPU 6	5	10	25	30 [14, 69]	535	-0.999

Pick	Configuration	p50	p99	p99.9	p99.9 pooled [95% CI]	Max	Cliff's delta
Random, seed 3	FIFO 76, CPU 1	5	10	61	63 [22, 82]	2,521	-0.998
Random, seed 4	FIFO 20, CPU 2	5	11	69	69 [44, 81]	8,414	-0.997

Table B.4: Budget study on the synthetic model, 30 seeds per cell. "Within 5%" is the fraction of seeds whose pick had a true 99.9th percentile no more than 5 per cent above the optimum. A12 is the probability that a pick of Bayesian optimisation has lower regret than a pick of random search; p is the one-sided Mann-Whitney test.

Budget	BO median regret	BO within 5%	BO exact optimum	Random median regret	Random within 5%	A12	p
10	17.5%	3%	0%	24.1%	3%	0.61	0.065
20	0.8%	83%	0%	14.6%	3%	0.96	< 0.001
40	0.3%	100%	43%	10.7%	20%	0.99	< 0.001
80	0.0%	100%	63%	6.4%	37%	1.00	< 0.001
150	0.0%	100%	77%	4.9%	53%	0.98	< 0.001

In the synthetic model of Section 4.7, a request's latency is the product of a body median and a log-normal factor with sigma body_sigma; with probability p (capped at 0.5) the request is also hit by a stall of scale times (1 + X), where X is Pareto-distributed with shape alpha. The eight tuning parameters act on the triple (body, p, scale) through the coefficients in Table B.5, which reproduces COEF_DEFAULTS in bo_sim.py exactly. The sensitivity study (Section 4.7) perturbs the 22 non-structural coefficients: magnitudes are multiplied by the factor f, and multiplicative effects are raised to the power f so that each stays on its side of 1.

Table B.5: Coefficients of the synthetic latency model. "Magnitude" and "effect" name the two perturbation rules of the sensitivity study; "fixed" marks the two structural thresholds that are not perturbed. The untuned default has p99.9 = 766.7 microseconds and the optimum of the 2,879,712-point space has p99.9 = 10.3 microseconds.

Coefficient	Value	Role in the model	Perturbation
base_median_us	5.4	Body median of the untuned default (microseconds)	magnitude
body_sigma	0.25	Log-normal sigma of the body	magnitude

Coefficient	Value	Role in the model	Perturbation
base_event_p	0.004	Stall probability p of the untuned default	magnitude
base_stall_us	320	Stall scale of the untuned default (microseconds)	magnitude
alpha	1.6	Pareto shape of a stall	magnitude
iso_decay	0.55	p multiplied by exp(-iso_decay x min(isolated_cores, iso_cap))	magnitude
iso_cap	4	Isolated cores beyond which isolation stops helping	magnitude
irq_noiso_p	1.6	p factor of IRQ affinity without isolation	effect
irq_iso_p	0.7	p factor of IRQ affinity with isolation	effect
irq_over_p	1.8	p factor of IRQ affinity with irq_over_min_iso or more isolated cores	effect
irq_over_min_iso	7	Threshold of isolated cores for irq_over_p	fixed
nohz_p	0.6	p factor of nohz_full with at least one isolated core	effect
rt_scale	0.45	Stall-scale factor of PREEMPT_RT	effect
rt_body	1.08	Body factor of PREEMPT_RT	effect
gov_p	0.8	p factor of the performance governor	effect
gov_body	0.97	Body factor of the performance governor	effect
numa_p	0.7	p factor of NUMA pinning	effect
numa_body	0.9	Body factor of NUMA pinning	effect
rtprio_scale_gain	0.3	Without isolation, scale multiplied by 1 - gain x ln(rt_priority) / ln(99)	magnitude
rt_high_p	1.3	p factor when rt_priority is rt_high_min or higher	effect
rt_high_min	95	Priority threshold for rt_high_p	fixed
coalesce_gain	0.3	p multiplied by 1 - gain x min(irq_coalesce_us, coalesce_sat_us) / coalesce_sat_us	magnitude
coalesce_sat_us	50	Coalescing interval at which its benefit saturates (microseconds)	magnitude
coalesce_body	0.04	Body increase per microsecond of coalescing (microseconds)	magnitude

Table B.6: Coverage of the one-at-a-time sensitivity sweep. Each cell gives, for the model in which one coefficient is perturbed by the factor f, the exact optimum p99.9 in microseconds and the lowest Vargha-Delaney A12 of Bayesian optimisation against random search over the budgets of 20 or more evaluations. The sweep ran the levels in the order 0.5, 2, 0.75, 1.5 and was stopped after 56 models, so the two extreme perturbations are complete for every coefficient and the 32 missing models are the milder ones. The lowest A12 in the table is 0.87 (iso_decay at f = 0.5, 0.865 before rounding); the base model has an optimum of 10.3 microseconds.

Coefficient	f = 0.5	f = 0.75	f = 1.5	f = 2
base_median_us	5.1 / 0.97	7.7 / 0.98	not run	20.6 / 0.95
body_sigma	7.0 / 0.97	8.5 / 0.97	not run	22.5 / 0.98
base_event_p	10.2 / 0.97	10.3 / 0.98	not run	10.4 / 0.93
base_stall_us	10.3 / 0.98	10.3 / 0.98	not run	10.3 / 0.98
alpha	10.3 / 0.94	10.3 / 0.98	not run	10.3 / 0.98
iso_decay	10.5 / 0.87	10.4 / 0.94	not run	10.2 / 0.95
iso_cap	10.5 / 0.90	10.4 / 0.94	not run	10.2 / 0.99
irq_noiso_p	10.3 / 0.98	10.3 / 0.97	not run	10.3 / 0.98
irq_iso_p	10.3 / 0.97	not run	not run	10.3 / 0.97
irq_over_p	10.3 / 0.98	not run	not run	10.3 / 0.97
nohz_p	10.3 / 0.94	not run	not run	10.3 / 0.97
rt_scale	10.3 / 0.93	not run	not run	10.3 / 0.93
rt_body	10.3 / 0.98	not run	not run	10.3 / 0.97
gov_p	10.3 / 0.98	not run	not run	10.3 / 0.97
gov_body	10.4 / 0.95	not run	not run	10.0 / 0.98
numa_p	10.3 / 0.98	not run	not run	10.3 / 0.97
numa_body	10.8 / 0.98	not run	not run	9.3 / 0.95
rtprio_scale_gain	10.3 / 0.96	10.3 / 0.97	not run	10.3 / 0.94
rt_high_p	10.3 / 0.98	not run	not run	10.3 / 0.96
coalesce_gain	10.3 / 0.97	10.3 / 0.98	not run	10.3 / 0.97
coalesce_sat_us	10.3 / 0.98	10.3 / 0.97	not run	10.3 / 0.95
coalesce_body	10.3 / 0.98	10.3 / 0.98	not run	10.3 / 0.98

Appendix C: Repository Contents

The project repository is public at https://github.com/cfpandrade/linux-tail-latency-tuning. It contains the measurement harness and search driver (scripts/experiments/real_pilot.py), the follow-up experiment (cstate_followup.py) and its analysis (cstate_analyse.py), the simulation model and studies (bo_sim.py, bo_budget.py, bo_sensitivity.py), the power analysis of the search comparison (real_power.py), the analysis script that produces every table and figure of Chapter 6 (real_pilot_analyse.py), the unit tests (test_real_pilot.py, which also holds the measurement smoke test, and test_bo_sim.py; tests/test_smoke.py checks only the repository layout and imports), the raw logs of every real measurement (data/real_pilot/run1/evals.jsonl and data/real_pilot/cstate1/evals.jsonl), and the simulation checkpoints (data/bo_sim_runs/). Rerunning the real study requires the rt-tests and stress-ng packages and a raised RLIMIT_RTPRIO and reproduces the design exactly, although measured values will differ with the hardware. Rerunning the simulation reproduces its results exactly, because every random draw is seeded.

Appendix D: Project Management and Deviations

The project was planned twice. The original proposal, written for the Research Methods module, scheduled fourteen weeks of cloud experiments on a cumulative gradient of six kernel configurations. In August, after it became clear that a cumulative gradient cannot attribute a change in the tail to any one mechanism, the plan was rewritten around Bayesian optimisation and compressed to eight weeks, working back from the final submission on 19 October, with all cloud measurements to finish by 21 September. Table D.1 sets the planned milestones against what happened, using the dates of the module calendar, the git history of the repository, and the timestamps in the experiment logs.

Table D.1: Planned milestones against the actual record. Dates before 13 October come from correspondence and the module calendar; dates from 13 October onwards come from git commits and experiment log timestamps.

Planned	Date	Actual	Date
Original proposal (cloud gradient, fourteen weeks)	July	Proposal reviewed; gradient design found unable to attribute effects	early August
Supervisor allocation and topic approval	week 1	Supervisor contacted and topic approved	20 August
Revised proposal and Ethical Considerations Form	24 to 31 August	Proposal finalised and ethics form submitted (all answers No)	31 August
Harness built, cloud instances provisioned, PTP verified	11 to 24 August	Measurement design corrected (ENA timestamps, KS test, RQ3)	early September
Cloud search and confirmation runs complete	21 September	Decision to measure on a local workstation and to add a simulation strand	12 October
Results and discussion drafted	28 September	Repository initialised; chapters scaffolded; local harness and first pilot	13 October
Draft implementation and results submitted	5 October	Simulation budget study (46 minutes) and sensitivity sweep started	14 October
Draft report submitted	12 October	Real study, 401 measurements in 3.6 hours; C-state follow-up; sweep analysed	15 October
Final report submitted	19 October	Body condensed to the word limit; repository made public; numerical, figure, citation, and style audits	18 October

Three deviations shaped the dissertation. First, the cloud study was not executed: the bare-metal instances, real-time kernel, and instance-hours it needed were not available within the project, so the empirical strand moved to a commodity workstation restricted to user-space mechanisms (Section 4.2), and the cloud design was retained as Appendix A with the revisions the local results suggest. Second, because a single host cannot validate a search procedure against a known answer, a simulation strand with an exactly known optimum was added (Section 4.7); it was run before the real study so that the simulation budget study could set the real budget. Third, nearly all measurement and writing fell into the final week: the two simulation studies ran on 14 and 15 October, the real study on the morning of 15 October, and the report was restructured on 18 October. The compression left no time for a second real-measurement campaign, which is why the C-state follow-up (Section 6.4) has five runs per condition and why the factorial phase has three. It also forced one priority rule when the sensitivity sweep and the real study competed for the same machine: real measurements took precedence, because a measurement not taken before submission cannot be regenerated, whereas the simulation is seeded and checkpointed, so the remaining 32 models resume from the published run directory, and the joint sweep starts, with the same command. The sweep was therefore stopped at 56 of 88 models, with the extreme perturbations complete (Table B.6). The repository records every step, so the sequence can be audited from the commit history.